Optimized
Plan

Web Application

G o o gl e

DNS

Database

Before Big Data Era

3 Vs Of Big Data

Web Application

G o o gl e

DNS

Database

Velocity
Volume
Variety

No

Monolithic (Vertical Scaling)

Server

Web Application

G o o gl e

DNS

RAM
Processor
ROM

Down

Distributed (Horizontal Scaling)

Server

Web Application

G o o gl e

DNS

Server Server

Server

Server

Server

RAM
Processor
ROM

Server

Server Server

Server

Server

Hadoop

HDFS

MapReduce

CSV

file-01.csv

1GB

Data

Storage

Without HDFS

CSV

file-01.csv

1GB
With HDFS

Server

Server

Server

Server

Server

Server

Server

Server

128 MB each

HDFS

MapReduce Map + Reduce

CSV

file-01.csv

CSV

file-01.csv

CSV

file-01.csv

CSV

file-01.csv

CSV

file-01.csv

CSV

file-01.csv

CSV

file-01.csv

CSV

file-01.csv

CSV

file-01.csv

CSV

file-01.csv

CSV

file-01.csv

CSV

file-01.csv

CSV

file-01.csv

CSV

file-01.csv

CSV

file-01.csv

CSV

file-01.csv

I love

Spark

I love

Hadoop

Program

I love

Big Data

Data Engineer

M1

M2

M3

M4

M5

M6

M7

M8

"I"

:

"1"
"love

"

:

"1"

"

spark"

:

"1"

"I"

:

"1"
"love

"

:

"1"

"hadoop

"

:

"1"

"

program

"

:

"1"

"I"

:

"1"
"love

"

:

"1"

"Big

"

:

"1"
"Data

"

:

"1"

"Engineer

"

:

"1"
"Data

"

:

"1"

"I"

: [1,1,1] R1

"love

"

: [1,1,1] R2

"

spark"

: [1] R3

"hadoop

"

: [1] R4

"

program

"

: [1] R5

"big

"

: [1] R6

.
.
.

"I"

:

"3"

"love

"

:

"3"

"

spark"

:

"3"

"hadoop

"

:

"3"

"

program

"

:

"3"

"big

"

:

"3"

Mappers & Reducers are just tasks (not the nodes).

Server

Server

Server

Server

Server

Server

Server

Server

CSV

file-01.csv

CSV

file-01.csv

CSV

file-01.csv

CSV

file-01.csv

CSV

file-01.csv

CSV

file-01.csv

CSV

file-01.csv

CSV

file-01.csv

CSV

file-01.csv

1GB

Computation goes to the data. Data doesn

't move.

"I"

:

"1"
"love

"

:

"1"

Stored Locally R1

Reducer pulls the
key-value pairs
(intermediate transformations)
from local storage

Blocks Mappers

Inter result

(Keys-Values) Shuffling Reducers

CSV

Final File

HDFS

Apache Spark

Apache Spark is just an alternative to Hadoop MapReduce.
HDFS is still there.

Apache Spark is a unified distributed computing engine which
allows us to process big data in parallel.

Apache Spark Architecture

Cluster - Group of inter-connected machines.

Node - Machine.

Spark
Submit

>_

Cluster Manager

Host Host Host Host Host

Cluster Size

W: 2 machines
[10GB]

D: 1 machine
[10GB]

Cluster Manager

M1 M2 M3

Master Slaves

M1

Cluster Manager

Step-1:

M4 M5

Application Master
10GB

Step-2:

M1

Application Master
10GB

Cluster Manager

Create 2 workers

W1

W2

Cluster Manager

Driver

Application Master
10GB

Step-3:

W1

W2

Cluster Manager

Driver

Application Master
10GB

Spark Contexts VS Spark Session

Spark context is the entry point to our Spark Application.
It resides in the driver node.

Spark Context SQL Context Hive Context

With Spark 2.0 we combined all of these contexts, and we
started using Spark Session.

Access Data Lakes With Spark

Azure Databricks

ADLS Gen2

Access Connector

Inside DRIVER & Worker Nodes

DRIVER NODE

Application Master Container

JVM MAIN ()

JVM

PySpark Main

Py4j

PySpark Driver Application Driver

Worker NODE

Python JVM MAIN ()
(Optional)

When you will be using the Python
User Defined Functions

Transformation & Lazy Evaluation

Narrow Transformation

Id Name Desc

1
2
3
4

aa

bb

cc

dd

....
....
....
....

Id Name Desc

1 aa...
2 bb....

Id Name Desc

3 cc...

B 4 dd....

ig Data (256MB)

128

128

FILTER
Only Even

Id Name Desc

2
4

bb
dd

....
....

This transformation is a type of transformation that doesn

't

share or exchange data between the partitions.

Wide Transformation

Id Name Desc

1
2
3
4
1 aa....

aa

bb

cc

dd

....
....
....
....

Id Name Desc

1 aa...
2 bb....

Id Name Desc

1 aa...
3 cc...
4 dd....

128

128

GROUPBY

On IDS

1 aa
2 bb

1 aa
3 cc
4 dd

Shuffling

1 aa
1 aa
2 bb

3 cc
4 dd

1 2
2 1

3 1
4 1

1 2
2 1

3 1
4 1

This transformation is a type of transformation that does
share or exchange data between the partitions.

Lazy Evaluation

Code

>_

Spark

APACHE

1) Select Cols
2) Filters
3) Grouping

Action

Wait For Action

display()

Spark Catalyst Optimizer

Code

>_

Spark

APACHE

1) Select Cols
2) Filters
3) Grouping

Code

>_

Unresolved

Logical Plan

Resolved

Logical Plan

Catalog Analysis

Optimized
Logical Plan

Optimized
Logical Plan

Physical Plans Cost Model

Picked

Physical Plan

Executors

Spark Ecosystem

RDDs

Structured APIs

Dataframes Datasets SQL

Structured
Streaming

Libraries

RDDs

Resilient Distributed Dataset

RDDs are immutable

(Fault Tolerant)

rdd1 = 100

rdd2 = rdd1 + 10

rdd3 = rdd2 * 10

In case of failure

[1,2,3,4,5]

[1,2] [3,4,5]

Doesn

't have any
Schema. Just the
data structure

The data itself

Why to write code in Dataframes then?

RDDs don

't have any schema.
That means we can

't apply any optimization.

We used to write our code in RDDs but

it's not very readable.

We have to tell HOW as well along with WHAT
while writing code for RDDs.

RDDs are not the real data, it's just the
logical partitions

Id Name Desc

1
2
3
4
1 aa....

aa

bb

cc

dd

....
....
....
....

ADLS Gen2

Id Name Desc
1 aa...
2 bb....

Id Name Desc
1 aa...
3 cc...
4 dd....

RDD-1

Id Name Desc
1 aa...
2 bb....

Id Name Desc
1 aa...
3 cc...
4 dd....

RDD-1

FILTER

Id Name Desc
1 aa...
2 bb....

Id Name Desc
1 aa...
3 cc...
4 dd....

RDD-2

Jobs, Stages, and Tasks

Job

We

'

re indirectly saying that this is our job. Please trigger it.

Stage

Different stages for different set of transformation.
It can combine similar transformations within one stage.

Tasks

Tasks are your units of parallelism

1 JOB

Stage-1

Repartition

1 Task

Stage-2

Select

2 Tasks

Filter

bcz we have 2
partitions now

Shuffle Write

Shuffle
Read

Stage-3

GroupBY

200 Tasks

bcz wide
transformation

Repartition VS Coalesce

Data

Partitions

Repartition

Wide Transform

Coalesce

Narrow Transform

PySpark Transformations

SPARK

Python

Data Read

CSV

CSVCSV

JSON
JSON
JSON

CSV

CSVCSV

JSON
JSON
JSON

Schema

Enforce
Schema

StructType

DDL Schema

SELECT

Select Only these 3 cols

Alias
Meaningful Name

Filter

Only Returned Orders
Scenario-1

Returned Orders OR

Cancelled Orders Scenario-2

Returned Orders OR
Cancelled Orders OR

Shipped Orders Scenario-3

isin ('Returned'

,

'Shipped'

,

'Cancelled')

withColumnRenamed

Rename this column

withColumn

This is the go-to API for either transforming the column
or adding/creating a new one.

Scenario-1

New Column

Scenario-2

Scenario-3

Total_Price

TypeCast

Table1 Table2

order_id (INT) order_id (STR)

Table1 Table2

order_id (STR) order_id (STR)

Sorting

Desc Asc

Limit

Table1

100 records

Drop

dropDuplicates

No Dups

Scenario

No Dups

Union & UnionByName

id name ph id name ph

DF1 DF2

id name ph

id name ph

Union

id name ph

ph name id

DF1 DF2

id name ph

id name ph

UnionByName

Date Functions

SQL
SQL Database

Internet

Folder

Date Date

String Functions
Upper Case

Handling Nulls

Split & Index

Split It

Delimiter

Explode
Exploded in the rows

Array Contains
Apply Filter On Array

Group BY

P117 18

Collect List

user product

1 p1
1 p2
2 p1
3 p2
3 p3

1 [p1,p2]
2 [p1]
3 [p2,p3]

Pivoting

c1
c2
c3
c4
c5

complete cancelled return

10 2 1
5 1 0

when-otherwise

Return Flag
low
low
mid
low
mid
low

.
.
.
.

JOINS

Table1 Table2

id ph em
1 0 a

2 9 b

oid c_id
101 1

102 3

Inner Join

Inner Join

Left Join

Inner Join

Rest of the Left

Right Join
Inner Join Rest of the right

Full Join

Inner Join

Plus Rest

Anti Join
Left Anti Join

Window Functions

Row_Number

Marks

33
45
99
99
100

RowNumber

1
2
3
4
5

Unique number to each record.

Rank VS Dense_Rank

Marks

33
45
99
99
100

Rank Dense

1
2
3
3
5

1
2
3
3
4

Special Functions

Year

2020
2021
2022
2023
2024

Revenue

100
110
140
90
150

Total

100
210
350
440
590

Year

2020
2021
2022
2023
2024

Revenue

100
110
140
90
150

Total

590
590
590
590
590

User Defined Functions

Custom Function

Concat & Concat_Ws

Data Writing

Folder

XXX

Transform
Folder

modes

Overwrite

MODES

Folder

Destination

New

Destination

Append

Folder

Destination

Folder

Destination

X

X
X

Error

Folder

Destination Destination

X
Error

Ignore

Folder

Destination Destination

X

Ignore

File Formats

Folder

XXX

Transform
Folder

CSV

X

Folder

XXX

Transform
Folder

JSON

JSON

Folder

XXX

Transform
Folder

parquet

Parquet

Delta Format

Folder

Parquet

- ACID properties

- Optimization/Indexing

- Commit & Rollback

- Structured Query Env

SQL Table Capabilites

Folder

Parquet

_delta_log

Delta Log/Transactional Log

- ACID properties

- Optimization/Indexing

- Commit & Rollback

- Structured Query Env

SQL Table Capabilites

Upsert/Merge

id cat

1 shoes

2 clothes

Folder

Parquet

_delta_log

id cat

2 food

3 food

Day-1

Day-2 UPSERT

Update + Insert

id cat

1 shoes

2 clothes

1 shoes

2 food

3 food

Parquet

_delta_log

delta_object

Target

id cat

2 food

3 food

Day-2

source_object

Source
UPSERT

UPSERT ARCHITECTURE

SparkSQL

Table

CSV

file-01.csv

JSON

file-01.json

SQL Skills Delta Lake

Temporary Views

CSV

file-01.csv

Dataframe Temporary view

SQL Commands

SparkSQL DFs

Temporary view

Dataframe

Global Temp View

Global Temporary Views are

"CLUSTER SCOPED" that means you can
access them from any notebook attached to the same cluster. Whereas,
Temporary Views are session-scoped which are not available in the other
notebooks bcz of new session.

DDL Commands

D Generic Table atabase

UPSERT SparkSQL

UPSERT With MergeInto API (Spark 4.0+)

JOINS Mechanism In Spark

Shuffle Sort Merge Join

id name

1 rahul

2 anna

201 ben

210 sim

6 tom

9 raj
101 priya
98 bob

32 rob

employee

id salary
1 1100

2 100

201 900

210 2100

6 3000

9 1200

101 750

98 800

32 1000

salary

1 rahul
201 ben
32 rob

2 anna

101 priya

210 sim
6 tom

98 bob

9 raj

1 1100
201 900
32 1000

2 100
101 750

210 2100
6 3000

98 800
9 1200

df1

df2

df1.join(df2,df1['id']==df2['id'],

'inner)

. . . 200

P1 P2

Hash(id=1) = A very big value
Hash(1) = 1 (For this explanation)

Hash(id)%n

1%200

1 rahul

1 1100

P10

210 sim

210 2100

201 ben

201 900

= 1

This way, it will create partitions so that same id of each dataframe
is available on the same partition.

P1

1 rahul

201 ben 1 1100

201 900

This is costly....

So we perform

"SORTING" before applying the final join.

P1

1 rahul 1 1100

201 ben 201 900

The

"SHUFFLING"

part is done....

Now, SORTING, BUT WHY Sorting?....

P1
1 rahul 1 1100
201 ben 201 900

Everything
Inside the Core

Executor

Shuffle Hash Join

Now, HASHING, BUT WHY Sorting?....

P1

1 rahul

201 ben 1 1100

201 900

In-Memory
Hash Table

O(1)

O(logn)

Broadcast Join

id name d_id
1 rahul 1

2 anna 2

201 ben 1

210 sim 3

6 tom 1

9 raj 3
101 priya 2
98 bob 2

32 rob 3

employee

d_id name

1 axz

2 xyz
3 abc

dept

This is very small (~5mb)

512 mb

Executor-1 Executor-2

1 rahul
201 ben
32 rob

2 anna

101 priya

210 sim
6 tom

98 bob

9 raj

df1
d_id name
1 axz

2 xyz
3 abc

df2

1 rahul
201 ben
32 rob

210 sim
6 tom

2 anna

101 priya

98 bob

9 raj

d_id name
1 axz

2 xyz
3 abc

Sharing Will
Take Place d_id name

1 axz

2 xyz
3 abc

Broadcasted Broadcasted

(No Shuffling)

Driver Memory Management

Code

>_ Spark

APACHE

- - driver-memory 2G

JVM Heap

spark.driver.memory

- scheduling
- metadata

- query plans
- .show()/.collect()
- broadcast

2G Overhead

10% or 384MB

spark.driver.memoryOverhead

- Objects Memory

2G + 10%/384MB

JVM HEAP

OVERHEAD

DRIVER

Driver OOM

JVM HEAP

OVERHEAD

DRIVER

Executors

Total Data = 10G

df.show()

only 1

JVM HEAP

OVERHEAD

DRIVER

Executors

Total Data = 10G

df.collect()

DRIVER OOM

Executor Memory Management

Code

>_ Spark

APACHE

- - driver-memory 2G
- - executor-memory 10G
- - num-executors 10

Heap Memory
(Main Memory of Executor)

10G Overhead Memory

spark.executor.memory

- all of the execution

spark.executor.memoryOverhead

- Non JVM tasks

10G+10%

OffHeap Memory

0 By default

PySpark Memory

spark.executor.offHeap.size

spark.executor.pyspark.memory

1G

HEAP

OVERHEAD

10G

1G

Executor

JVM HEAP (10G)

Reserved Memory

Spark Pool Memory

User Memory

300 MB (For spark engine itself.)

60% (10G-300MB ~ 9.7G) = 5.8G

40% (10G-300MB ~ 9.7G) = 3.9G

- UDFs created
- Custom Data Structures

- Spark Internal Metadata

Spark Pool Memory/Spark Unified Memory (5.8G)

Storage Memory

Execution Memory

spark.memory.storageFraction

50% (5.8G) = 2.9G

50% (5.8G) = 2.9G

- Used For Caching

- Used For Actual Execution

Spark Unified Memory

Execution Memory Storage Memory

Scenario-1

Execution memory needs more memory to process the data.

Execution Memory Storage Memory

Scenario-2

Now, you want to cache some more data.

Execution Memory Storage Memory

Can

't evict this data

LRU

Scenario-3

Now, you want to cache some more data.

Execution Memory Storage Memory

Scenario-4

Now, you want to execute more data

Execution Memory Storage Memory

LRU

Scenario-4

Now, you want to execute more data

Execution Memory Storage Memory

Data Spilling & Executor OOM

Data Spill

When Executor can

't keep the data in RAM anymore,
it moves the data to the

"DISK"

Executor OOM

Why Executor OOM when executor can spill the data to the disk?

id name
1 aa
1 aa
1 aa
1 aa
1 aa
1 aa
1 aa
2 bb

10GB

Execution Memory Storage Memory Spill To Disk

Drive

Execution Memory

Storage Memory

New Data

New Data

In-Use Blocks

Can

't evict

Spill To Disk

Drive

Highly Skewed
(id=1 istaking 90% of 10GB)

df.groupBy('id').agg(count('name'))

partition-1: id=1 (9G)

file(9GB) Drive

5.8GB (Pool Memory)

Executor
OOM

Edge Node

Submit

>_

Cluster Manager

Senior Dev

Without Edge Node

With Edge Node

Submit

>_

Cluster Manager

Edge Node

Submit

>_

Submit

>_

Deployment Modes

Submit

>_

Cluster Manager

Edge Node

Submit

>_

Submit

>_

Cluster Mode

Submit

>_

Cluster Manager

Edge Node

Submit

>_

Submit

>_

Cluster

Cluster

Cluster

Cluster

Driver

Client Mode

Submit

>_

Cluster Manager

Edge Node
Submit

>_

Cluster

Driver

Client

Cluster

AQE - Adaptive Query Execution

This was added to Spark 3.0+ as an

"OPTIMIZATION LAYER".

- Dynamically COALESCE the partitions.

- Dynamically optimizing the SKEWNESS.

- Dynamically improves the JOIN STRATEGY.

COALESCE

id category
1 food
2 clothes
1 food
1 food

3 gadget
3 gadget
1 food
4 ess
5 member
6 flower

df.groupBy("category").agg(count('id'))

food

flower

ess

mem

gadget

clothes

Without AQE

200 Partitions

- 6 occupied
- 194 empty

Problems

- 194 empty tasks
- Data won't process in a
uniform manner.

- 199/200 tasks completed

food

flower ess

mem gadget clothes

With AQE

Coalesced

Coalesced

Advanvtages

- 3 total tasks

- 0 empty tasks.
- All tasks will be

completed together
(almost)

SKEWNESS

id category
1 food
2 clothes
1 food
1 food

3 gadget
3 gadget
1 food
4 ess
5 member
6 flower

df.groupBy("category").agg(count('id'))

id = 1 has 80% of the DATA

food

flower

ess

mem

gadget

clothes

flower ess

mem gadget clothes

Coalesced

Coalesced

food

food

food

Split

With AQE

- The skewed data should be
at least 5 times the median.
- The size of the skewed data
should be at least 256MB.

JOIN Strategy

DF1 DF2

10GB 8GB

Sort-Merge Join Physical Plan

AQE Enabled

DF1 DF2

10GB 8GB

Sort-Merge Join Physical Plan

DF1

10GB

DF2

5MB

During Runtime

Broadcast Join

Cache & Persist

CACHE

Mobile

Lorem ipsum dolor sit amet,
consectetur adipiscing elit,sed
do eiusmod tempor incididunt
ut labore et dolore magna
aliqua. Ut enim ad miveniam,
quis nostrud exercitaullamco
laboris nisi ut aliquip ex ea
ommodo consequat. Duis aute
irure dolor in reprehenderit i

Lorem ipsum dolor s,
consectetur adipng d
do eiusmopor incididunt
ut labore et dolore magna
aliqua. Ut enim ad miveniam,
quis nostrud exercitaullamco
laboris nisi ut aliquip ex ea
ommodo consequat. Duis aute

df1 = spark.read.format("csv")\
. load("path")

df2 = df1.withColumn("flag"

,lit(0))

Short Term Memory

df1 will be referred using the above code.Spark
will recreate the df1 using the DAG.

df3 = df1.filter(col('id')==1)

df1 = spark.read.format("csv")\
. load("path")

df2 = df1.withColumn("flag"

,lit(0))

df3 = df1.filter(col('id')==1)

After Cache
df1

Storage Memory

PERSIST

Persist helps us to persist the data in memory and disk (as per our needs)

Let's say, df1 data size is bigger than the storage memory. Then,
Executor will try to store as much data as it can in the storage memory,
and rest of the data will be either RECOMPUTED or SPILLED to disk.

Memory_Only Memory_And_Disk

CACHE is the special type of PERSIST

- For Dataframes: (MEMORY_AND_DISK)
- For RDDs: (MEMORY_ONLY)

STORGAE Levels

MEMORY_ONLY

df.persist(StorageLevel.MEMORY_ONLY)

MEMORY_AND_DISK

df.persist(StorageLevel.MEMORY_AND_DISK)

df.cache()

DISK_ONY

df.persist(StorageLevel.DISK_ONLY)

MEMORY_ONLY_2

df.persist(StorageLevel.MEMORY_ONLY_2)

MEMORY_ONLY_SER

MEMORY_AND_DISK_SER

OFF_HEAP

spark.memory.offHeap.enabled = True

df.persist(StorageLevel.OFF_HEAP)

- Store serialized data.

- Helps in GC overhead.
- Helps in Persist.

Serialized & Deserialized
between PYTHON - JVM

Partitions & Pruning

Partitions

orders

ADLS Gen2

Spark

APACHE

parquet
parquet

Without Partitions

orders

ADLS Gen2

Spark

APACHE

With Partitions

year=2025

month=1

parquet

.
.
.
.
.
.
.
.

Partition Pruning

orders

ADLS Gen2

year=2025

month=1

parquet

month=1

parquet

Dynamic Partition Pruning

orders

year=2025

month=1

parquet

fact_orders
dim_date

date

parquet

parquet

SELECT

*

FROM
fact_orders
LEFT JOIN
dim_date
ON fact_orders.dim_date_key = dim_date.dim_date_key
WHERE

dim_date.year = 2025

orders

year=2025

month=1

parquet

fact_orders

dim_date

date

parquet

parquet

File Scan

File Scan

Dynamic Filter

Broadcast Exchange

JOIN

Salting For Skewness

id cat

1 food
1 food
1 food
1 food
1 food
1 food
1 food
2 clothes
2 clothes

3 plants
3 plants

GROUPBY

partition_1

partition_2

partition_3

id cat salt

1 food 1
1 food 1
1 food 1
1 food 2
1 food 3
1 food 3
1 food 3
2 clothes 1
2 clothes 2

3 plants 1
3 plants 1

After Salting

GROUPBY

Salting In Joins

id prod

1 food
1 food
1 food
1 food
1 food
1 food
1 food
2 clothes
2 clothes

3 plants
3 plants

id cat

1 edible

2 lifestyle
3 nature

id cat salt

1 food 1
1 food 1
1 food 1
1 food 2
1 food 3
1 food 3
1 food 3
2 clothes 1
2 clothes 2

3 plants 1
3 plants 1

Add Salt Column

id cat salt

1 edible [0,1,2]
2 lifestyle [0,1,2]
3 nature [0,1,2]

Adding Array

id cat salt

1 edible 0
1 edible 1
1 edible 2

2 lifestyle 0
2 lifestyle 1
2 lifestyle 2
3 nature 0
3 nature 1
3 nature 2

Explode

SQL Hints

Code

>_

Spark

APACHE

SQL Hints

Broadcast Variable

This is a READ-ONLY shared variable that is broadcasted to all the executor nodes

{
"1"

:

"food"

"2"

:

"

electronics

"

"3"

:

"

clothes

"

}

Garbage Collection (GC)

Garbage Collection is a JVM process that re-claims the memory by
removing the unused objects.

(Spark Application creates a lot of temporary objects for shuffling, caching,
serialization etc.)

When the JVM runs Garbage Collection (GC), it stops the application
threads - STW Pause (Stop The World). That means, no computation
takes place during that time.

Pauses In GC

- Minor GC (Young Gen)
(Few Milliseconds)

- Major GC (Old Gen)
(Few Seconds)

Reduce GC

Use Kryo Serialization

This is much more faster and compact than Java Serialization.

spark.conf.set("

spark.serializer

"

,

"

org.apache.spark.serializer.KryoSerializer)

Avoid Caching

Control Partition Size

Tune JVM Memory

Avoid Shuffling

Tune GC Algorithm

- -conf "spark.executor.extraJavaOptions=-XX:+UseG1GC"

Executor

's Role In GC

Host

Small Executor

GC frequency will be high but pause will be low

Host

Big Executor

GC frequency will be low but pause will be high

Bucketing

o_id p_id

1 10
2 21
3 15
4 13
5 9
6 10
7 21
8 23

GroupBY

Number of Buckets = Total Size/128MB

- We use Bucketing for HIGH-CARDINALITY columns.
- It eliminates the SHUFFLING of partitions.

p_id%3

10
13
10

B0 B1 B2

21
15
9
21

23

GroupBy

No Shuffling Needed

(Bcz the whole BUCKET will go to each executor)

df.write.format("parquet")\
. mode("append")\
. bucketBy(3,

"product_id")\
. option("path"

,

".....")\
. saveAsTable("products_table")

Spark Structured Streaming

Source

JSON

file-01.json

JSON

file-01.json

JSON

file-01.json

Spark

APACHE

Delta Lake

Near-Real Time

Streaming Evolution

Record-At-A-Time Model

E1

E2

E3

Spark Streaming (DStreams)

E1

E2
E3

Spark Structured Streaming

Challenges:
- had to write different code for streaming & batch.
- No Coding Optimization

PROS:
- unified API

Programming Model Of Spark Structured Streaming

Unbounded Input Table
New Streams

Spark

APACHE

Incrementalization: User defines a batch-like query on top of
the unbounded input table, Structured Streaming
will automatically convert your batch-like query
to Streaming Execution Plan.

Spark

APACHE

Query

result

input table

t=1

t=2

Query

result

t=3

Query

result

Stateless VS Stateful Transformations

Unbounded Input Table

Stateless Transformation

Filter

Delta Lake

When you don

't need to use the result of

the previous streams.

Stateful Transformation

Unbounded Input Table

GroupBy

Delta Lake

When you need to use the results of the previously
processed streams.

Stateful Transformation Technical

E1

E2

E1

E2

2

2

2

2

Stream - 1 It is cached in the
executor memory

E1

E2

Stream - 2

E1

E2

2

2

2

3

+ 2
+ 2

+ 2

+ 2

= 4
= 4

= 4

= 5

Checkpoint Directory

Source

JSON

day1.json

Spark

APACHE

Delta Lake

parquet

JSON

day2.json
parquet

Checkpoint

sources

offsets

commits

metadata

Output Modes

Append

It is only available for Stateless Transformations

Source

Blue

Green

Red

Destination

Blue

Green

Red

Output

Blue

Green

Red

Maroon Maroon Maroon

Streams

Complete

It will return all the rows every time we trigger.

Source

Blue

Green

Red

Destination

Blue-1

Green-1

Red-1

Output

Blue - 1

Green-1

Red-1

Red
Blue - 1

Green-1

Red - 2

Blue-1

Green-1

Red-2

Update

It will only output those rows which are updated.

Source

Blue

Green

Red

Blue-1

Green-1

Red-1

Output

Blue - 1

Green-1

Red-1

Red Red-2 Blue-1

Green-2

Red-2

Green Green-2

Trigger Modes

Default

It will automatically trigger the next stream as soon
as the previous one is complete.

processingTime

We define at what interval spark needs to trigger the job.

once

Source

JSON

day1.json

Spark

APACHE

Delta Lake

JSON

day2.json

JSON

day1.json

JSON

day1.json

It will load all the data once and will stop the stream.
- It is used in backfilling and initial loads.

availableNow

It will load all the data once and will stop the stream.
- It is used in backfilling and initial loads.

continuous

It continuously runs your spark stream (instead of micro
batches).

Archive Source Files

Source

JSON

day1.json

Spark

APACHE

Delta Lake

Archive

parquet

JSON

da2.json

parquet

JSON

da3.json

ForEach Batch

input_df fx
Function

input_df DELTA LAKE

W Folder

ithout ForEachBatch

DELTA LAKE

Folder

DELTA LAKE

Folder

With ForEachBatch

Event Time VS Processing Time

2026-02-10 20:00

Event Time

2026-02-10 20:02

2026-02-10 20:05

2026-02-10 20:10

A PA C H E

2026-02-10 20:01

Processing Time

DELTA LAKE

Folder

Window Operations

Unbounded Input Table

Window

Now, we will perform aggregation
on top of the groupBy Key PLUS
the window column as well

Tumbling Window

It is a non-overlapping fixed size window.

Window = 10 mins
Trigger = 5 mins

W1

20:00 20:05 20:10
W2
20:10

20:15

W3

20:20 20:25 20:30

20:20

20:02
Green
Red

20:07
Green
20:05

Red-1
Green-1

20:10

Red-1
Green-2

20:12
Green
Blue
20:15

Red-1
Green-2

Green-1
Blue-1

Sliding Window

It is an overlapping but fixed size window

W1

20:00 20:05 20:10

W2

20:10

20:15

W3

20:20 20:25 20:30

20:20

20:02
Green
Red

20:07
Green
20:05 20:10

20:12
Green
Blue
20:15

Window = 10 mins
Trigger = 5 mins
Sliding Window = 5

Red-1
Green-1

Red-1
Green-2

Green -1

Red-1
Green-2

Green -2
Blue-1

Green -1
Blue-1

Session Window

It is non-fixed window interval

Initiate it with gap of 5mins.

20:00

20:04
Green
Red

20:09

20:09
Green
Red

20:14

w1

20:15
Green
Red

20:20

w2

Handling JSON

JSON

file-01.json Generic Table

A PA C H E

File Source Options

dir1

parquet

CSV

file-01.csv

dir2

parquet

Spark

APACHE

ignoreCorruptFiles

test_corrupt_df0 = spark.read.option("ignoreCorruptFiles"

,

"true")\
.parquet("examples/src/main/resources/dir1/"

,

"examples/src/main/resources/dir1/dir2/")

ignoreMissingFiles

pathGlobFilter

df = spark.read.load("examples/src/main/resources/dir1"

,

format="parquet"

, pathGlobFilter="*.parquet")

recursiveFileLookup

recursive_loaded_df = spark.read.format("parquet")\
.option("recursiveFileLookup"

,

"true")\
.load("examples/src/main/resources/dir1")

modifiedBefore & modifiedAfter

df = spark.read.load("examples/src/main/resources/dir1"

,

format="parquet"

, modifiedBefore="2050-07-
01T08:30:00")

df = spark.read.load("examples/src/main/resources/dir1"

,

format="parquet"

, modifiedAfter="2050-06-
01T08:30:00")

W1

20:00 20:05 20:10
W2
20:10

20:15

W3

20:20 20:25 20:30

20:20

20:02
Green
Red

20:07
Green
20:05

Red-1
Green-1

20:10

Red-1
Green-2

20:12
Green
Blue
20:15

Red-1
Green-2

Green-1
Blue-1

Late Arrival

20:08
Green

Late Arrival

Watermarks

By 11:00 PM

A MAX(col('event_time') - 10 minutes)

ccepted =

20:01
Blue

Late Arrival

df_str = spark.readStream.table('sparkcatalog.raw.stream_source')\
.groupBy('color'

,window('event_time'

,

'10 minutes'

,

'5 minutes')).agg(count('color').alias("count"))

df_str = spark.readStream.table('sparkcatalog.raw.stream_source')\
.groupBy('color'

,session_window('event_time'

,

'5 minutes')).agg(count('color').alias("count"))

Spark Connect

Spark Connect introduced De-Coupled client-server architecture.

Code

>_

VS Code

A PA C H E

Unresolved

Logical Plan

Client

gRPC

gRPC

Server

df = spark.read.format("csv")
display(df)

Spark Connect In Databricks - Databricks Connect

Code

>_

VS Code

Unresolved

Logical Plan

Client

df = spark.read.format("csv")
display(df)

databricks

Databricks Connect is built on top of OPEN-SOURCE SPARK CONNECT.

Databricks Connect

Pandas API On Spark

pandas

Python

X
file-01.excel

X
file-01.excel

X
file-01.excel

X
file-01.excel

X
file-01.excel

X
file-01.excel

X

file-01.excel X
file-01.excel

X
file-01.excel

X
file-01.excel

Traditional Pandas (Running On One Machine)

pandas

Python

Host

Host

Host

Host

Pandas API On Spark

Advantages
- Same Pandas Syntax (easy migration)

- Handles Big Data
(bcz normal Pandas run on single machine, whereas,
Pandas on Spark runs on multiple machines)

Host

X
file-01.excel

X
file-01.excel

X
file-01.excel

X
file-01.excel

X
file-01.excel

X
file-01.excel

X

file-01.excel X
file-01.excel

X
file-01.excel

X
file-01.excel

Pandas UDFs are BETTER.....

Spark

APACHE

Only PySpark Code

Code

>_

Driver

Python

Python

Java

Java

Py4J

Worker

Physical Plan

Only PySpark

PySpark + UDF

Spark

APACHE

PySpark + UDF

Code

>_

Driver

Python

Python

Java

Java

Py4J

Worker

Physical Plan
Python Java

Java

Java

Java

PySpark + Pandas UDF

Spark

APACHE

PySpark + Pandas UDF

Code

>_

Driver

Python

Python

Java

Java

Py4J

Worker

Physical Plan
Python Java

Java

Python

Java

Java

Serialization

Deserialization

Python Java

Java

Arrow Arrow

Python UDF Pandas UDF (Vectorized)

PROBLEMS

- Heavy Serialization

- Row-By-Row Execution

- Python Function called
millions of times

SOLUTIONS

- Transforms in batches (10k rows by default)

- Convert column to Arrow Format

- Python function calls are fewer

Types of Pandas UDF (Vectorized)

Series To Series

@pandas_udf("double")
def price_func(col1:pd.Series) -> pd.Series:
return col1 * 10

Partition

100k Rec

.
.
.

10k

10k

10k

10k

1

2

Iterator To Iterator

@pandas_udf("double")
def price_func(col1:Iterator[pd.Series]) -> Iterator[pd.Series]:

model = ml_model()

for batch in col1:
return ml_model(batch*10)

Partition

100k Rec

10k

10k

10k

10k

Multiple Iterator Series To Iterator Series

Series To Scalar

Spark Submit

wordcount.py

Python

Spark

APACHE

spark-submit is the command-line tool that is used to submit a
spark application to a cluster

spark-submit \
- - master yarn \
- - deploy-mode cluster \
- - num-executors 4 \

- - executor-memory 4G \
- - executor-cores 2 \

wordcount.py

Delta Optimization

OPTIMIZE COMMAND

Delta Lake

parquet

parquet

parquet

parquet

parquet

parquet

_dellta_log
Delta Lake

parquet

_dellta_log

parquet

OPTIMIZE orders;

ZORDERBY COMMAND

select * from orders
where id=1

Delta Lake

parquet

parquet

parquet

parquet

parquet

parquet

_dellta_log
Delta Lake

parquet

_dellta_log

parquet

OPTIMIZE orders
ZORDER BY id;

select * from orders
where id=1

Liquid Clustering

Delta Lake

parquet

parquet

parquet

parquet

parquet

parquet

_dellta_log

CLUSTERBY id;

Logical Clusters

Data Skipping

Accumulators

These are WRITE-ONLY shared variables that are used for aggregartion.

Executors can add/update the values.

Driver will read the value.

This is mainly used for tracking bad records.

spark.sparkContext.longAcculumator("badrecords_var")

NEXT

NEXT

In-Memory Columnar Storage

- spark.sql.inMemoryColumnarStorage.compressed:
When set to true, Spark SQL automatically selects a compression codec for each column based on data statistics.

- spark.sql.inMemoryColumnarStorage.batchSize:
Controls the size of batches for columnar caching to improve memory utilization, though large sizes risk out-of-memory errors.

File-Based Source Tuning

- spark.sql.files.maxPartitionBytes:
Sets the maximum number of bytes to pack into a single partition when reading file-based sources like Parquet, JSON, and ORC.

- spark.sql.files.openCostInBytes:
Estimates the cost to open a file in bytes to help Spark decide how to pack multiple small files into a single partition.

- spark.sql.files.minPartitionNum:
Provides a suggested minimum number of split file partitions for file-based sources.

- spark.sql.files.maxPartitionNum:
Provides a suggested maximum number of split file partitions, triggering Spark to rescale partitions if the initial number is too high.

- spark.sql.sources.parallelPartitionDiscovery.threshold:
Configures the threshold of input paths required to enable parallel file listing via a distributed Spark job.

- spark.sql.sources.parallelPartitionDiscovery.parallelism:
Configures the maximum parallelism used when listing job input paths.

Shuffle and Join Configuration

- spark.sql.shuffle.partitions:
Configures the number of partitions to use when shuffling data for joins or aggregations.

- spark.sql.autoBroadcastJoinThreshold:
Configures the maximum size in bytes for a table to be broadcast to all worker nodes during a join; set to -1 to disable.

- spark.sql.broadcastTimeout:
Sets the timeout in seconds for the wait time during broadcast joins.

Adaptive Query Execution (AQE)

- spark.sql.adaptive.enabled:
The umbrella configuration to enable or disable Adaptive Query Execution.

- spark.sql.adaptive.advisoryPartitionSizeInBytes:
The target size of shuffle partitions during adaptive optimizations like coalescing or splitting skewed partitions.

- spark.sql.adaptive.coalescePartitions.enabled:
When enabled with AQE, Spark will coalesce small contiguous shuffle partitions to avoid excessive small tasks.

- spark.sql.adaptive.coalescePartitions.parallelismFirst:
When true, Spark ignores the advisory partition size to respect a minimum partition size and maximize parallelism.

- spark.sql.adaptive.coalescePartitions.minPartitionSize:
Defines the minimum size of shuffle partitions after coalescing has occurred.

- spark.sql.adaptive.coalescePartitions.initialPartitionNum:
Sets the initial number of shuffle partitions before coalescing begins.

- spark.sql.adaptive.optimizeSkewsInRebalancePartitions.enabled:
Enables the optimization and splitting of skewed shuffle partitions during rebalance operations.

- spark.sql.adaptive.rebalancePartitionsSmallPartitionFactor:
A factor used to determine if a partition is small enough to be merged during a splitting operation.

- spark.sql.adaptive.autoBroadcastJoinThreshold:
Configures the maximum table size for broadcasting specifically within the AQE framework.

- spark.sql.adaptive.localShuffleReader.enabled:
Allows Spark to use a local shuffle reader when shuffle partitioning is not strictly required.

- spark.sql.adaptive.maxShuffledHashJoinLocalMapThreshold:
Configures the maximum size per partition allowed to build a local hash map to prefer shuffled hash joins over sort-merge joins.

- spark.sql.adaptive.skewJoin.enabled:
Enables dynamic handling of data skew in sort-merge joins by splitting skewed partitions into evenly sized tasks.

- spark.sql.adaptive.skewJoin.skewedPartitionFactor:
A multiplier used with the median partition size to identify if a partition is skewed.

- spark.sql.adaptive.skewJoin.skewedPartitionThresholdInBytes:
The minimum byte size a partition must exceed to be considered skewed.

- spark.sql.adaptive.forceOptimizeSkewedJoin:
Forces the optimization of skewed joins even if it necessitates an extra shuffle.

- spark.sql.adaptive.optimizer.excludedRules:
Allows users to specify a comma-separated list of optimizer rules to disable within AQE.

- spark.sql.adaptive.customCostEvaluatorClass:
Defines a custom class for cost evaluation during adaptive execution.

Storage Partition Join

- spark.sql.sources.v2.bucketing.enabled:
Enables shuffle elimination by leveraging partitioning reported by compatible V2 data sources.

- spark.sql.sources.v2.bucketing.pushPartValues.enabled:
Attempts to eliminate shuffles when one join side has missing partition values from the other.

- spark.sql.requireAllClusterKeysForCoPartition:
Requires join or MERGE keys to match the partition keys exactly to eliminate a shuffle.

- spark.sql.sources.v2.bucketing.partiallyClusteredDistribution.enabled:
Enables skew optimizations for V2 bucketing by partially clustering one side of a join.

- spark.sql.sources.v2.bucketing.allowJoinKeysSubsetOfPartitionKeys.enabled:
Attempts to avoid shuffles even if join conditions do not include every partition column.

- spark.sql.sources.v2.bucketing.allowCompatibleTransforms.enabled:
Attempts to avoid shuffles if partition transforms are compatible but not identical.

- spark.sql.sources.v2.bucketing.shuffle.enabled:
Attempts to avoid shuffle on one side by recognizing the V2 data source partitioning on the other side.

SparkSQL Properties

SparkSQL Properties

Apache Spark Bootcamp

Spark

APACHE